

Funciones

en C

Modularización · Reutilización · Diseño elegante

Clase 08 · 26/05/2026

Ing. Behringer · Ay.Prof. Capia

```
int suma(int a, int b)
{
    return a + b;
}
```

Beneficios de modularizar



Modular los Sistemas



Limitar el margen de error



Mejorar la Calidad



Legibilidad y Organización



Reusabilidad



Mantenibilidad



Abstracción



Ahorrar espacio en Memoria



Reducir Repetición de Código

¿Qué lenguajes usan funciones?

C

C++

Java

Python

JavaScript

Go

TypeScript

Rust

!! Todos !!

Podés definir una función de tal manera que pueda ser llamada desde cualquier parte del programa

C

```
void mi_funcion();
int main() {
    mi_funcion();
    return 0;
}
void mi_funcion() {
    printf("Hola!\n");
}
```

JavaScript

```
function miFuncion() {
    console.log("Hola!");
}

miFuncion();
```

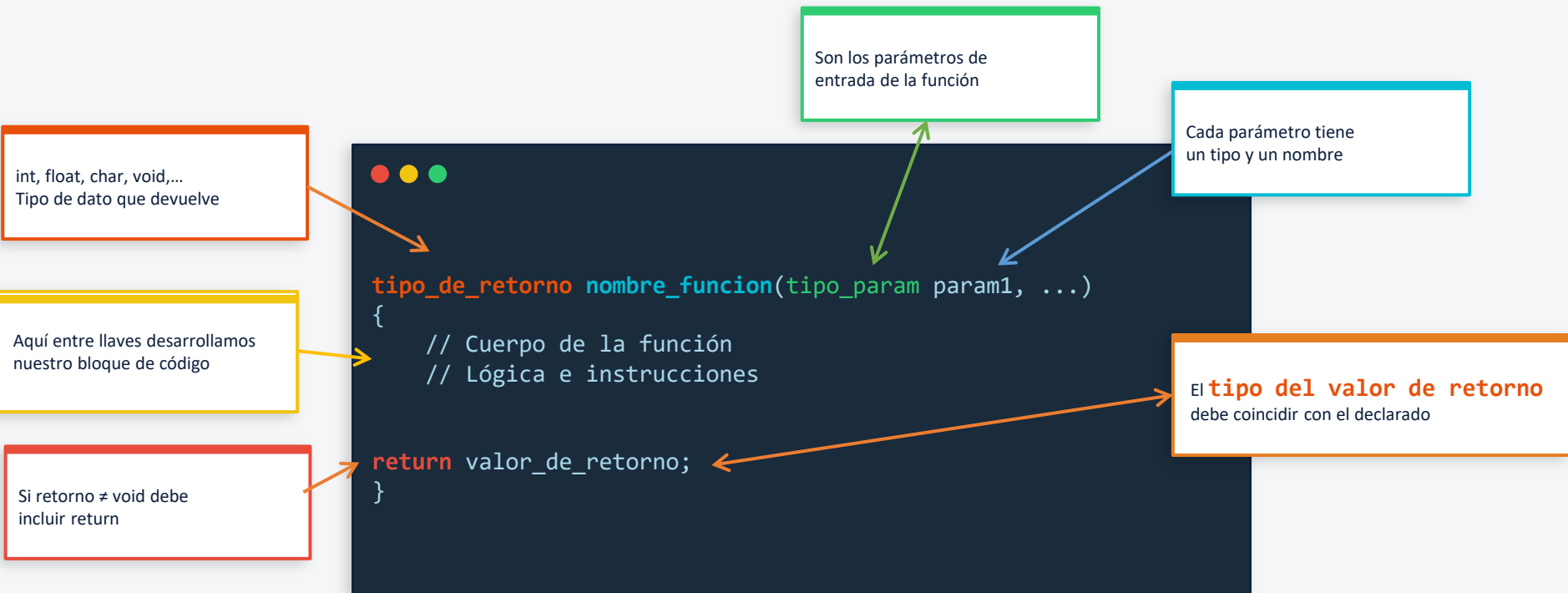
Go

```
func miFuncion() {
    fmt.Println("Hola!")
}

func main() {
    miFuncion()
}
```

Python

```
def mi_funcion():
    print("Hola!")
mi_funcion()
```



⚡ **Una función que calcula NO imprime**
Separaremos responsabilidades, es clave en el diseño de una función

La declaración de una función se denomina **PROTOTIPO**

Tipo de retorno

int, float, void,...

Nombre

Descriptivo, lo que hace

Parámetros (;)

Tipos y nombres, cerrado con punto y coma

El compilador usa los prototipos para validar que los tipos de datos al llamar la función coincidan con los de la definición.

```
● ● ●  
#include <stdio.h>  
  
float Area_rectangulo(float, float);  
float entrada(char *);
```

Problema: Calcular el área de un rectángulo dado su base y altura.
Fórmula: $\text{área} = \text{base} \times \text{altura}$. Base y altura no pueden ser menores a cero.

```
● ● ●
#include <stdio.h>
float calcularArea(float base, float altura);

int main(void) {
    float base, altura, area;

    printf("Base: "); scanf("%f",&base);
    printf("Altura: "); scanf("%f",&altura);
    area = calcularArea(base, altura);
    printf("Area: %.2f\n", area);
    return 0;
}
```

```
/* Definición — después del main */
float calcularArea(float base, float altura)
{
    return base * altura;
}
```

① Prototipo

Declara ANTES del main.
El compilador lo necesita.

② main llama

Pasa argumentos.
Recibe el resultado.

③ Definición

Implementa la lógica.
Valida que base y altura > 0.

④ return

Devuelve el resultado.
Tipo coincide con el prototipo.

PASAJE POR VALOR

- Los argumentos se pasan por valor
- Para pasar variables a las funciones
- La función recibe una COPIA del dato

 *Analogía: le das una FOTOCOPIA.
Puede escribir en ella sin tocar el original.*

vs

PASAJE POR REFERENCIA

- Los argumentos pasan por referencia
- Permite que la función modifique el original
- Para devolver valores vía argumento

 *Analogía: le das las LLAVES.
Puede entrar y modificar el original.*

01

Función Factorial

La función factorial se define como:

Factorial(n) = 1 si n = 0 Factorial(n) = n × Factorial(n-1) si n > 0

 Escribir la función Factorial y un programa que la llame varias veces para distintos valores de n.

Firma de la función

```
long factorial(int n);  
  
/* llamadas en main */  
printf("%ld", factorial(5));
```

Salida esperada

factorial(0)	= 1
factorial(1)	= 1
factorial(5)	= 120
factorial(10)	= 3628800

02

Combinaciones — equipo de trabajo

Hacer un programa que calcule de cuántas formas diferentes se pueden seleccionar un equipo de 3 personas de un grupo de 10 candidatos.

¡Para ellos utilizamos la fórmula!

$$C(n, k) = n! / (k! \times (n - k)!)$$

$$n = 10$$

Total de candidatos

$$k = 3$$

Personas a seleccionar

$$C(10,3) = 10! / (3! \times 7!)$$

= 120 formas diferentes

```
#include <stdio.h>
void TerminarPower(char *msj);
int main(void)
{
    TerminarPower(“FIN”);
    return 0;
}

void TerminarPower(char *msj);
{
    Printf(“%-30s”,mensaje);
}
```